



Github과 함께하는 AI 기반 개발 플랫폼

October 24th 2025



Soeun Park

AI&Apps Solution Engineer

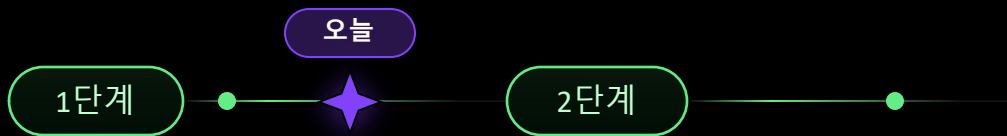


목차

- 01 Github 기반의 AI와 함께하는 개발 사이클
- 02 주요 기능 및 로드맵
- 03 Demo
- 04 시나리오 논의



새로운 시대의 제품 개발



인간 우선



인간 + 에이전트



GitHub Copilot 은 2021년에
생성형 AI 운동에 불을 지폈습니다. 🔥

그 후 프론티어 모델은 전례 없는 속도
로 발전하고 있습니다.

개발자는 에이전트를 오케스트레이션하여
개발 플로우를 제어합니다.

더 많은 코드는 '코드' 만 많아지는 것이 아닙니다.

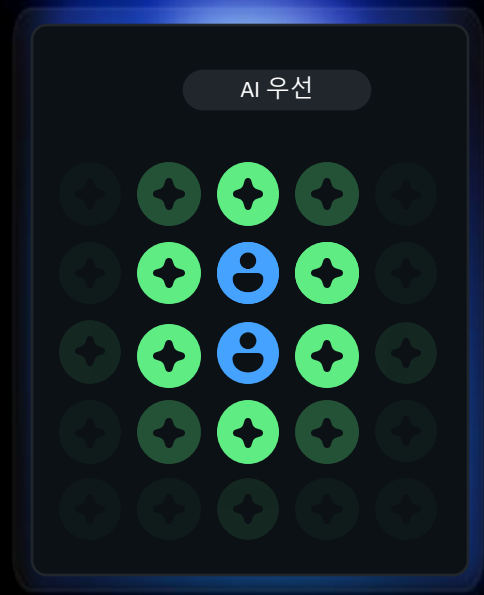
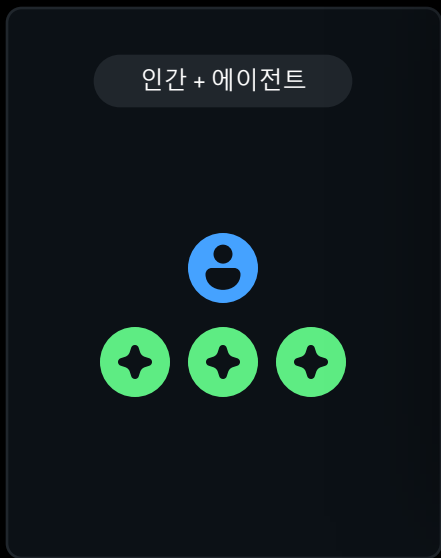
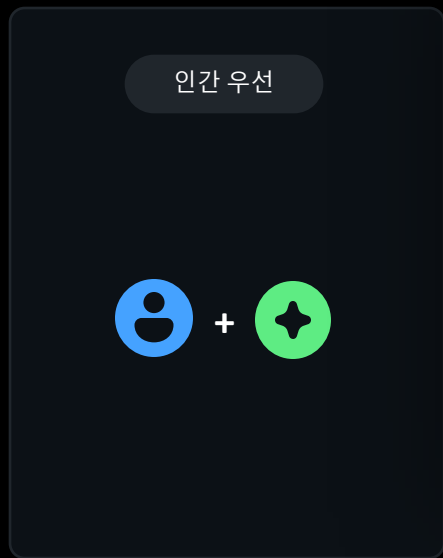
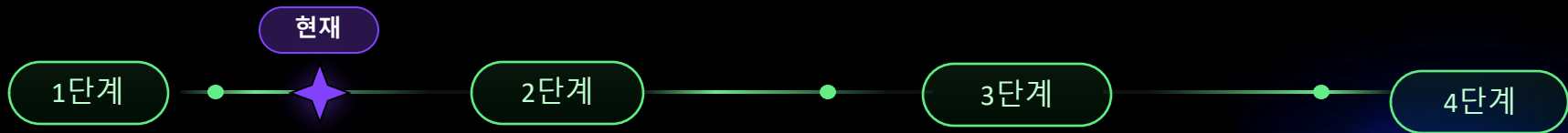


인간 우선



인간 + 에이전트









브레인 스토밍

계획

코딩

보안 및 품질

인간의 승인

SRE

```
mock.patch.object(MyClass, 'my_generator_method') as mock_method:
    mock_method.return_value = iter(['x', 'y'])
    # Now, when an instance of MyClass calls my_generator_method,
    # it will receive
```

```
mock.patch.object(MyClass, 'my_generator_method') as mock_method:
    mock_method.return_value = iter(['x', 'y'])
    # Now, when an instance of MyClass calls my_generator_method,
    # it will receive
```

```
mock.patch.object(MyClass, 'my_generator_method') as mock_method:
    mock_method.return_value = iter(['x', 'y'])
    # Now, when an instance of MyClass calls my_generator_method,
    # it will receive
```

```
mock.patch.object(MyClass, 'my_generator_method') as mock_method:
    mock_method.return_value = iter(['x', 'y'])
    # Now, when an instance of MyClass calls my_generator_method,
```

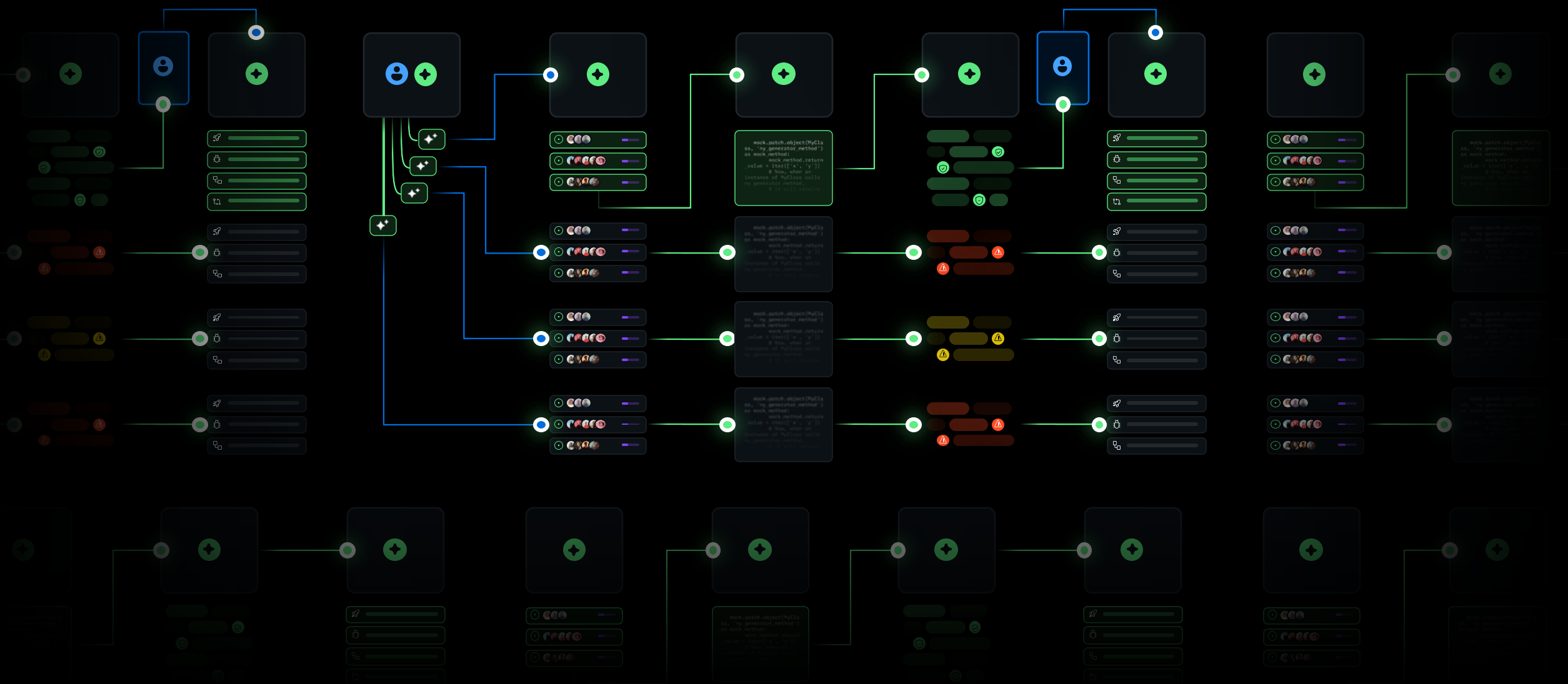
📅 계획

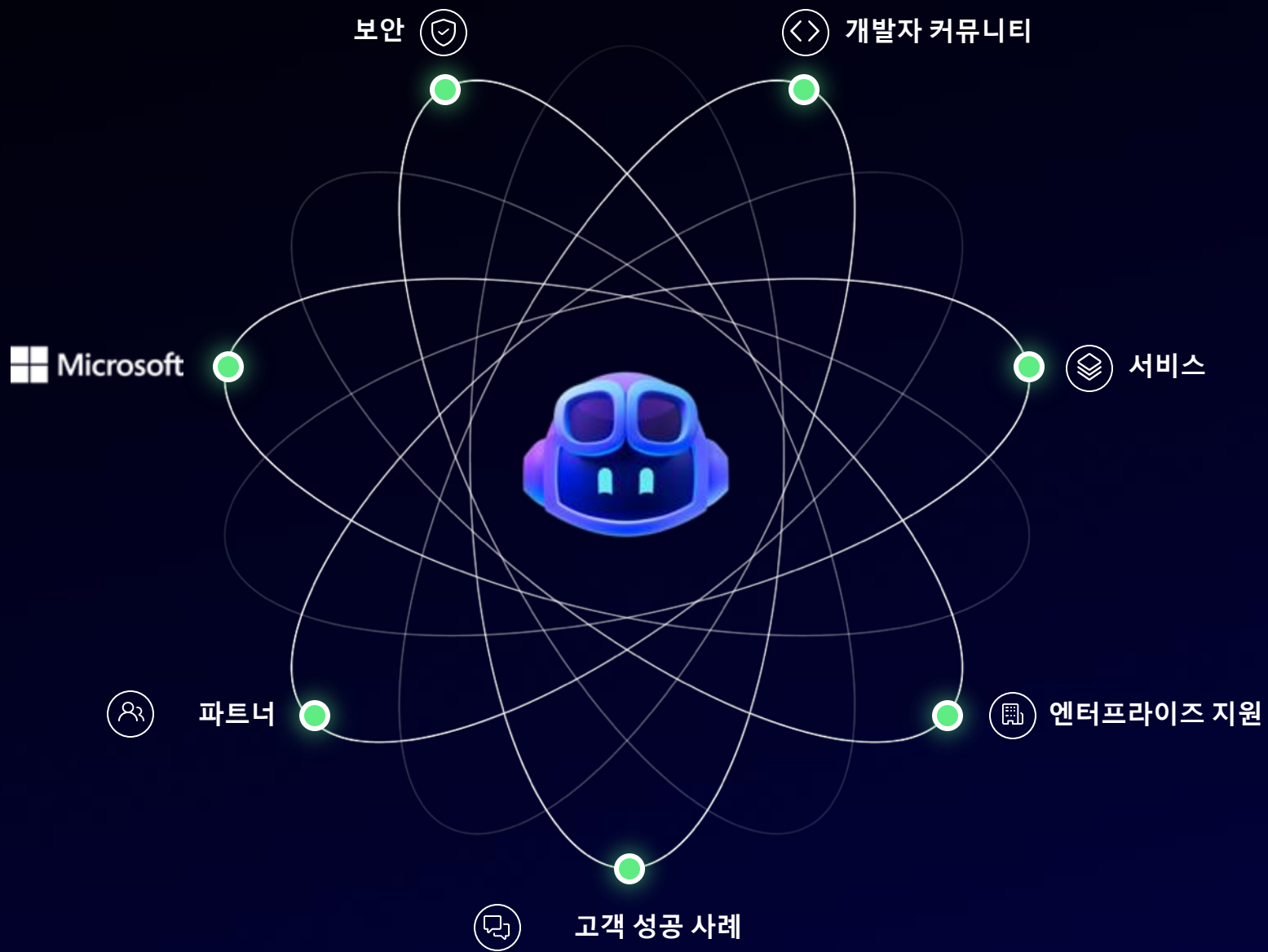
📄 코드

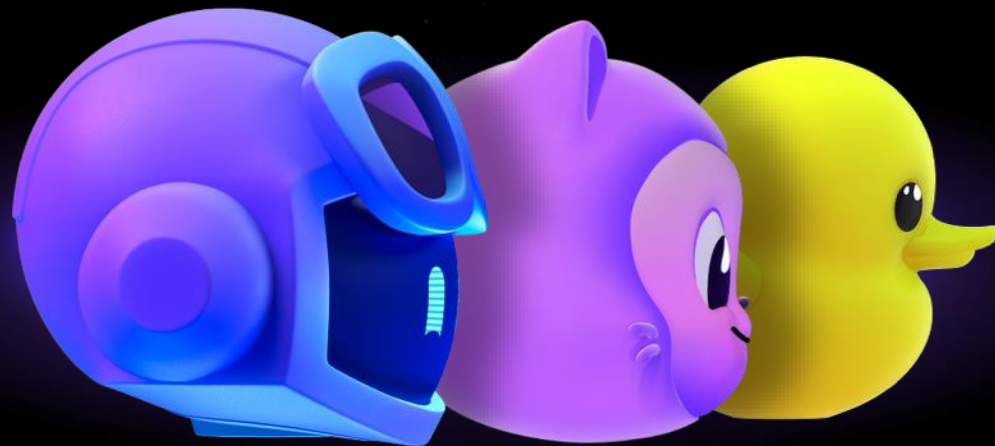
🕒 리뷰

🛡️ 테스트 및 보안

🚀 운영







전 세계 개발자를 위한 AI 기반 개발자 플랫폼

1억 5천만+ 개발자

Fortune 100대 기업의 90%

4억 2천+ 리포지토리



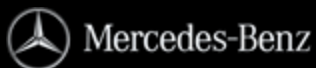
stripe



intel.



PHILIPS



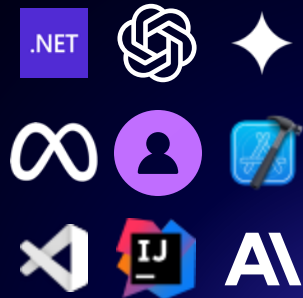
P&G



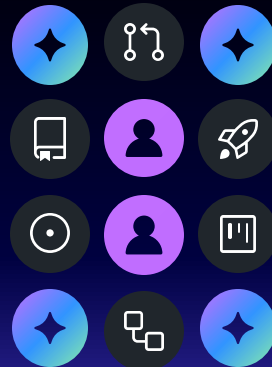
AI 시대에 오신 것을 환영합니다

GitHub의 차이점

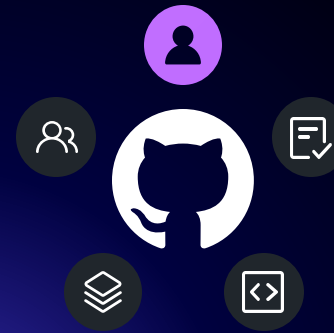
개발 환경 선택



협업



지식



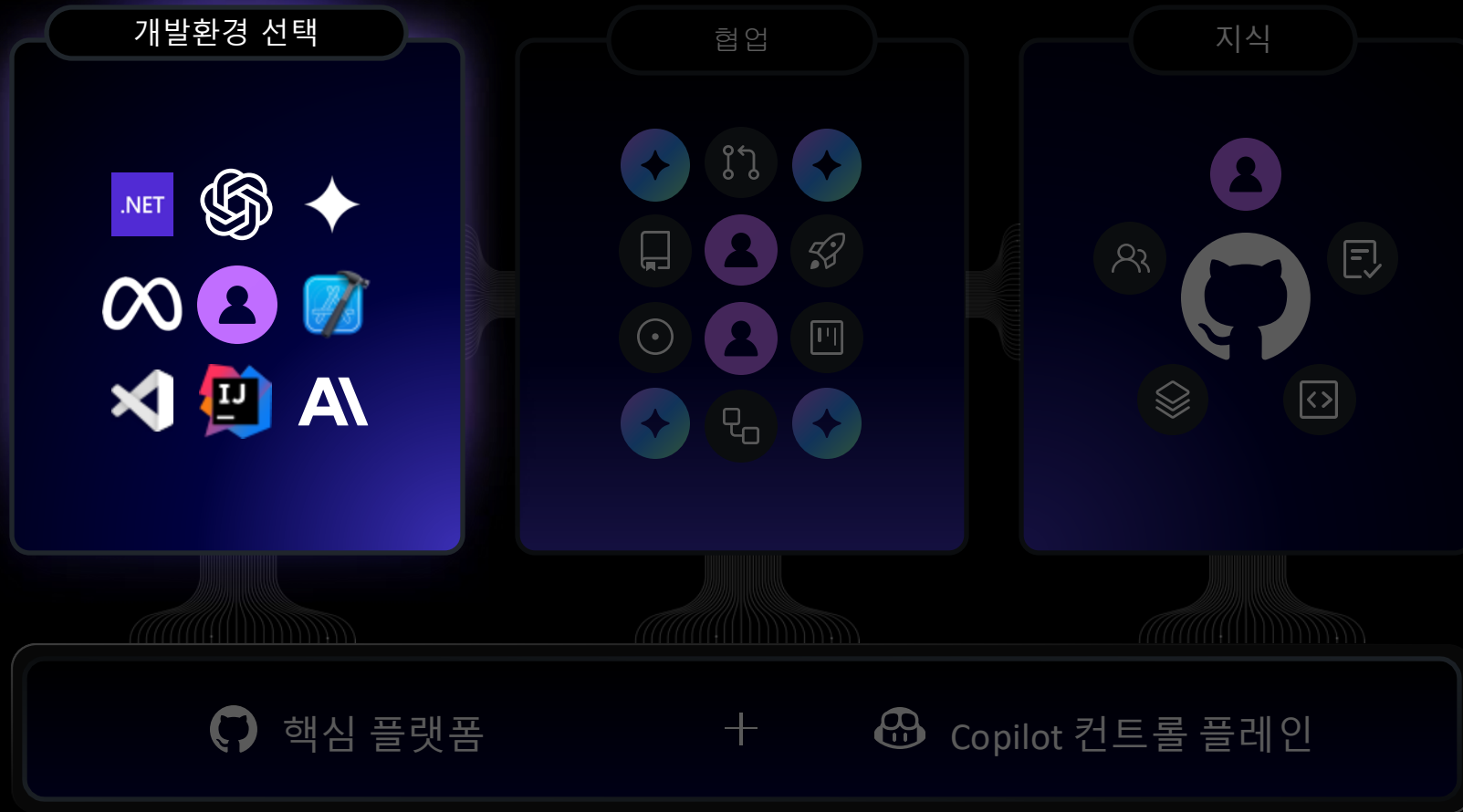
 핵심 플랫폼

+

 Copilot 컨트롤 플레인

AI 시대에 오신 것을 환영합니다

GitHub의 차이점



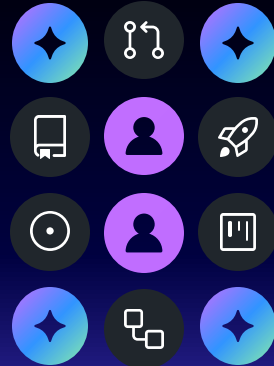
AI 시대에 오신 것을 환영합니다

GitHub의 차이점

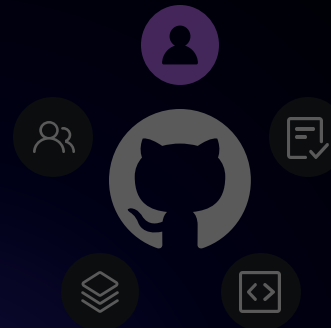
개발환경 선택



협업



지식



 핵심 플랫폼

+

 Copilot 컨트롤 플레인

AI 시대에 오신 것을 환영합니다

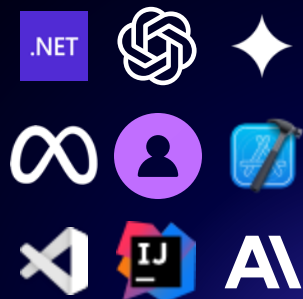
GitHub의 차이점



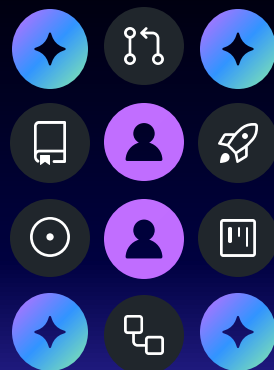
AI 시대에 오신 것을 환영합니다

GitHub의 차이점

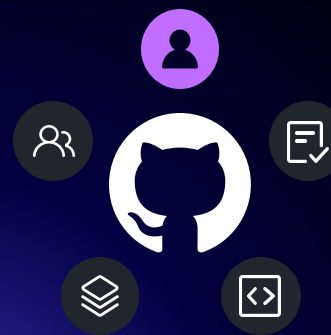
개발환경 선택



협업



지식



 핵심 플랫폼

+

 Copilot 컨트롤 플레인

Copilot 통합 기능

AI 기반 개발자 플랫폼

정책 및 거버넌스

📅 계획

계획 에이전트

문제

공간

<> 코드

에이전트 모드

코딩 에이전트

Github Spark(preview)

🔍 확인

자동 수정

코드 검토

PlayWright

풀 리퀘스트

🛡️ 전개

작업

Spark runtime

AI 워크플로

🚀 운영

운율학

모델

SRE 에이전트

ESSP



통합 및 MCP 연동

🧑‍🔬 에이전트 모드

새로운 수준의 자율성을 제공하는 Copilot Agent 모드

✓ 의도 감지

✓ 자가 치유 기능

✓ 런타임 오류 자동 해결

Working Set (3 files)

Accept

Discard



index.css public/css

default.css public/themes

<> index.ejs src/views

+ Add Files...

Add the functionality to search for competitions by name. Add the necessary tests. Run the tests to make sure everything looks good.



Agent ▾

GPT 4o ▾



✓ Agent

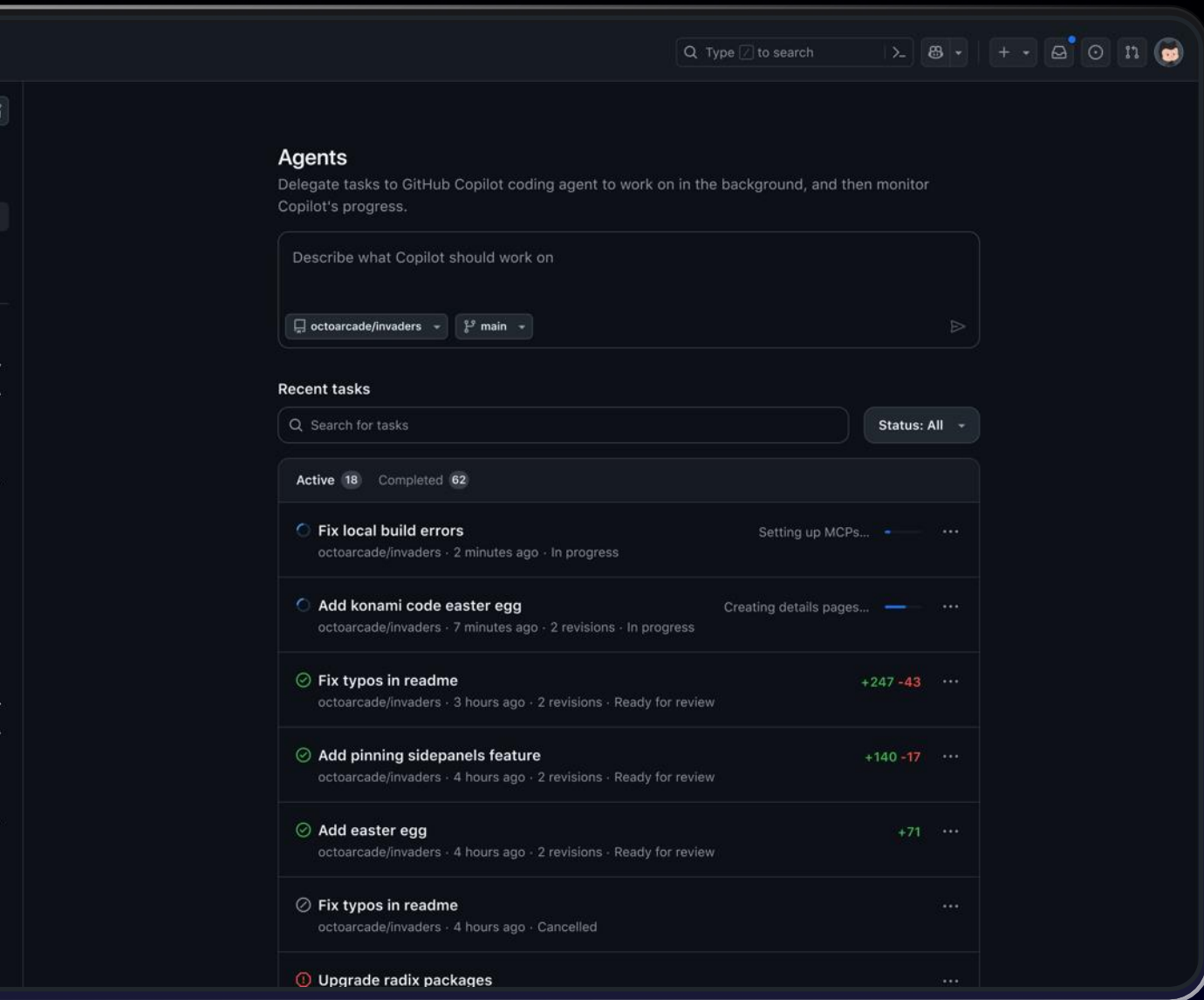
Edit

✓ Visual Studio

✓ Eclipse

✓ JetBrains

✓ XCode

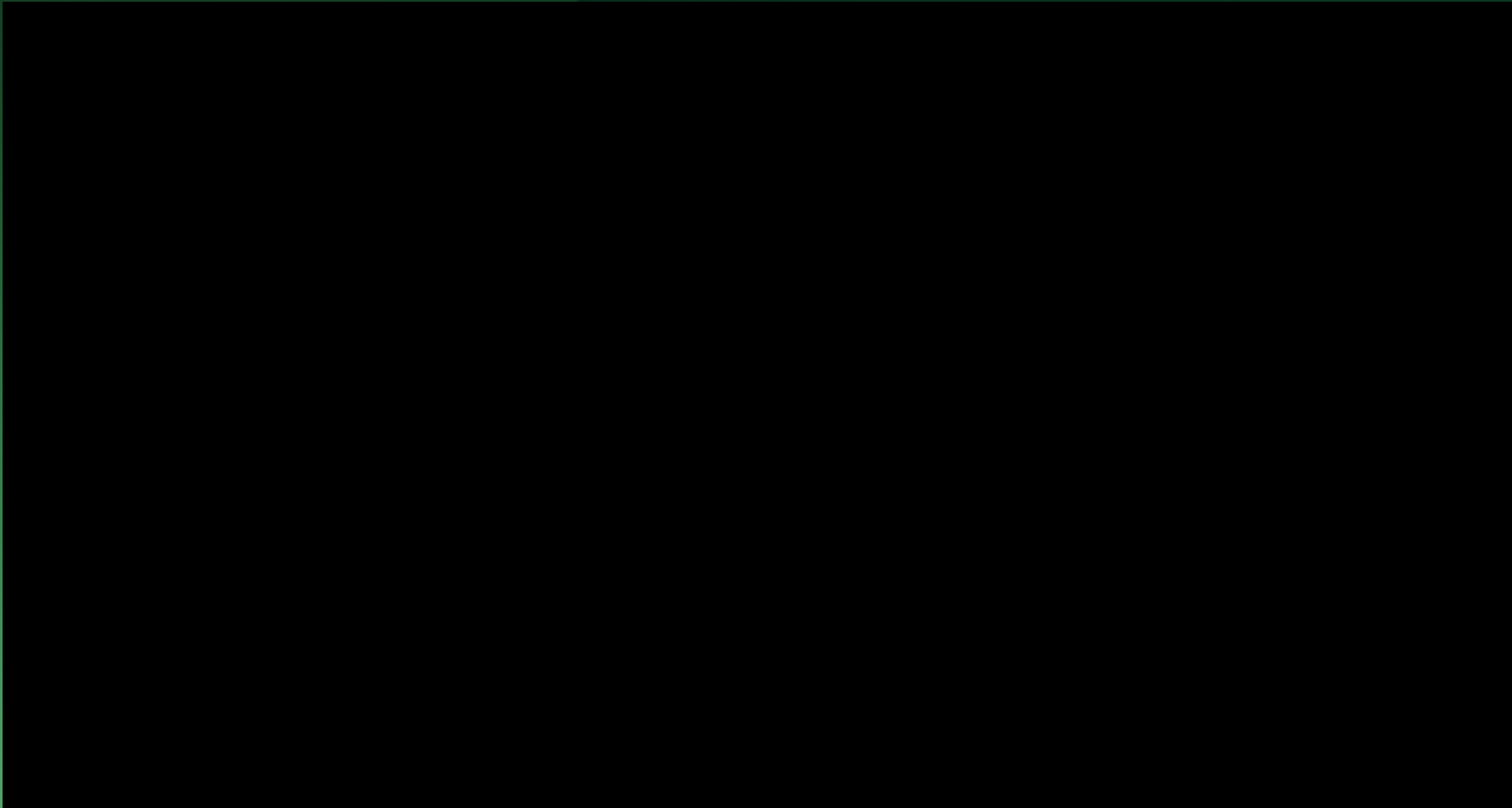


📅 계획

에이전트 패널

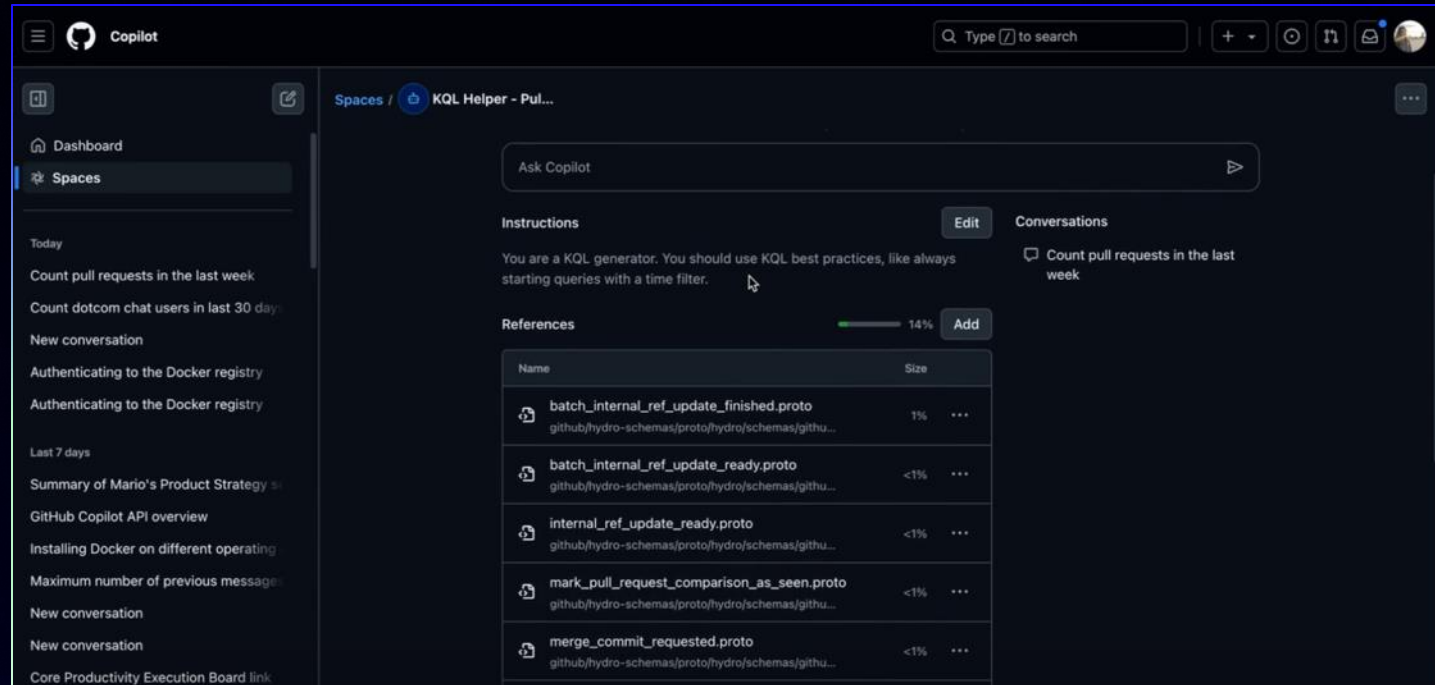
AI 기반 개발자를 위한 커맨드센터
[Github.com/copilot](https://github.com/copilot)

여러 Repository/작업물에 걸친 답변과 제안을 받는 “Copilot Space”





Copilot space



✓ 코드, KQL 쿼리, 테스트 사례, 문서 생성

✓ 글쓰기 스타일 정의

✓ 일관성 있는 공동 작업

 코드

CLI 기반으로 Copilot 사용하기



```
Skylars-MacBook-Pro:runtime skylar-anderson$ npm run start:cli -- --banner
```


MCP.. 써야 한다는데, 뭔가요?



Anthropic에서 첫 소개

개방형 프로토콜



AI가 세상에 접근하는 방식

Tool calling



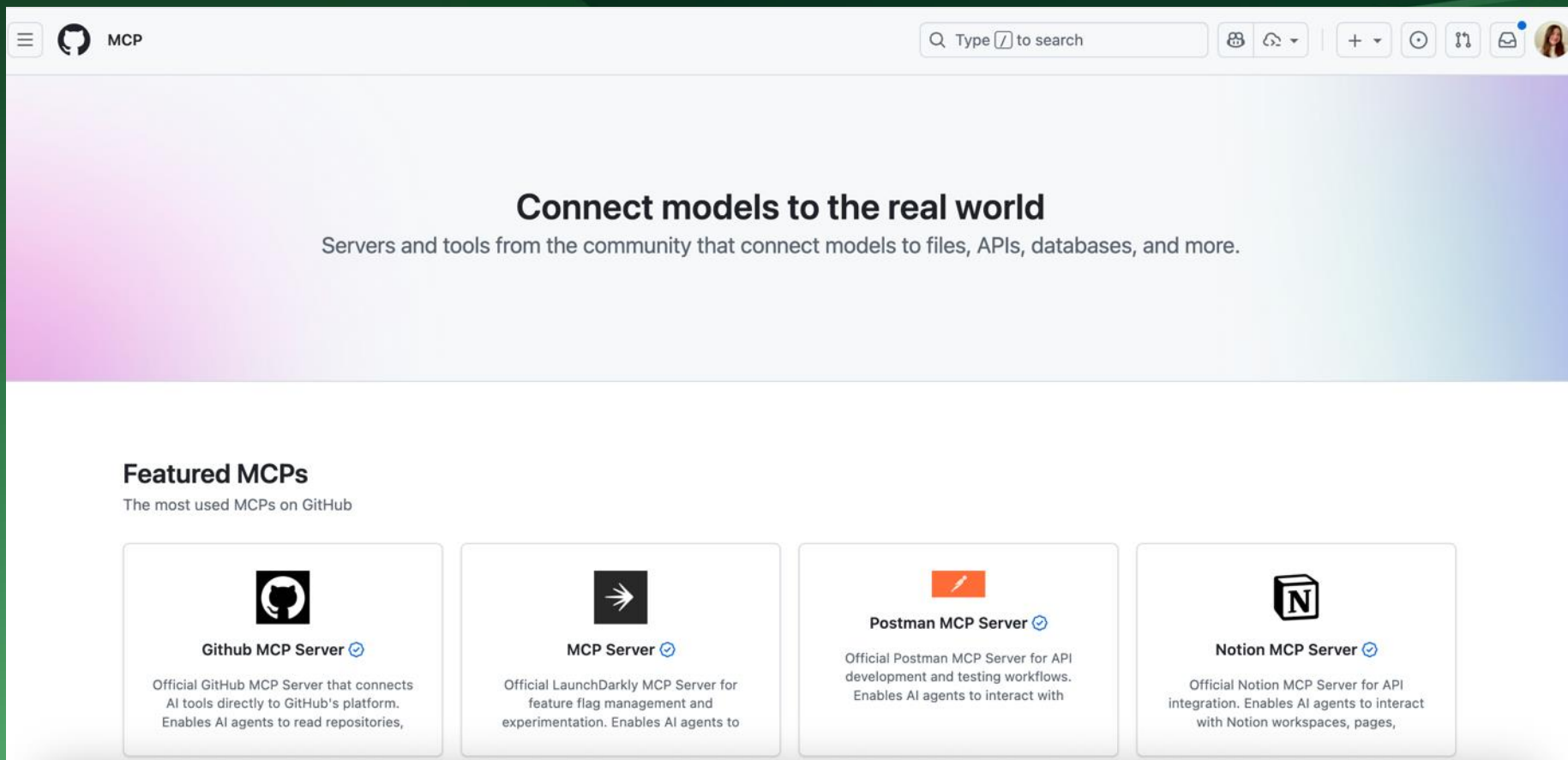
로컬 및 원격

두 종류의 MCP 서버:
컴퓨터의 로컬, 클라우드의 원격

📅 MCP 레지스트리

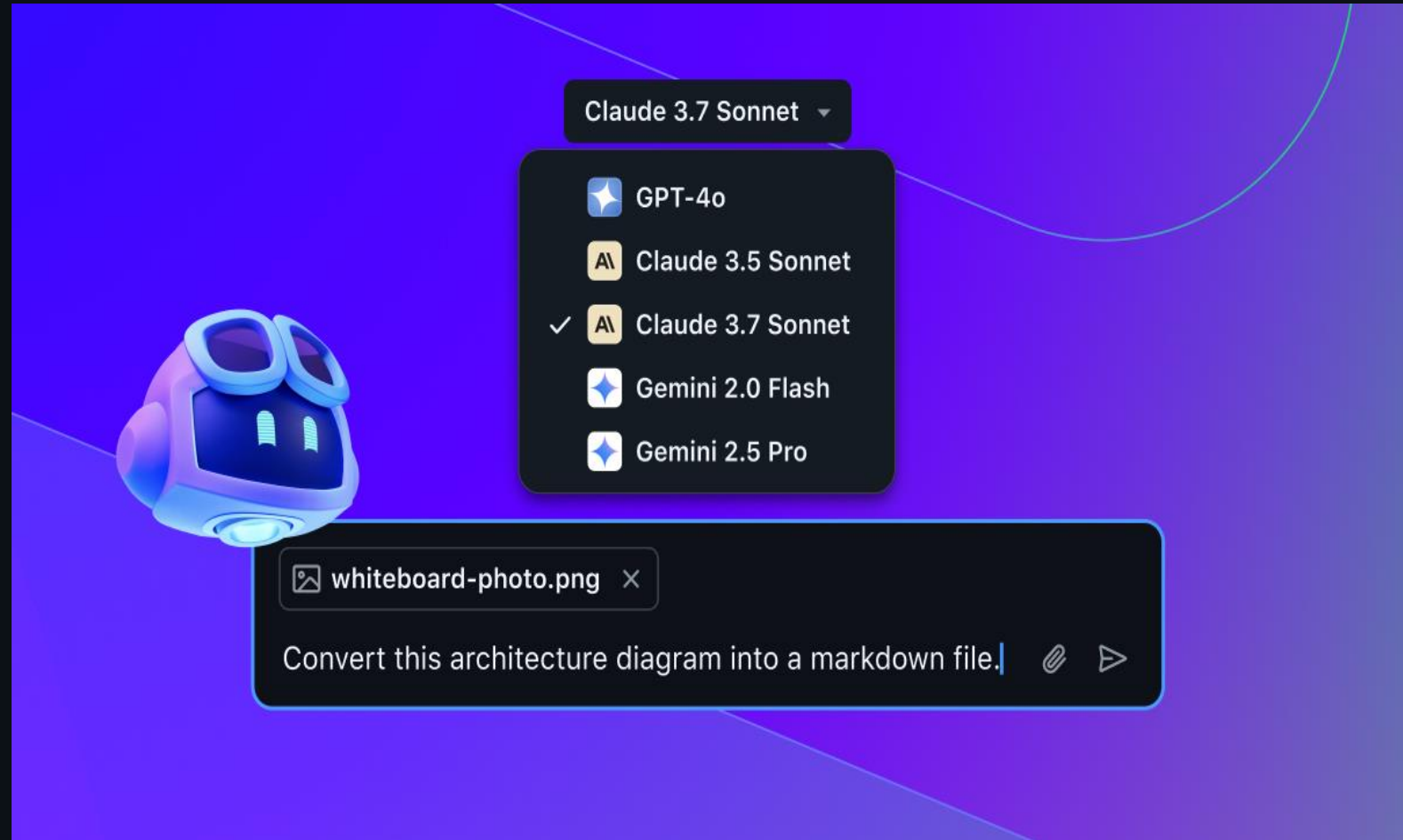
GitHub MCP 레지스트리

MCP 호환 제품에 MCP 서버를 쉽게 설치



Vision

- Copilot Chat에 그림파일을 첨부하여 코드 제안
- 에러 스크린샷을 첨부하고 Copilot으로 부터 수정을 위한 코드를 제안 받아 문제 해결
- Claude Sonnet 3.5, Claude Sonnet 3.7, Gemini 2.0 Flash, Gemini 2.5 Pro, and GPT-4o models

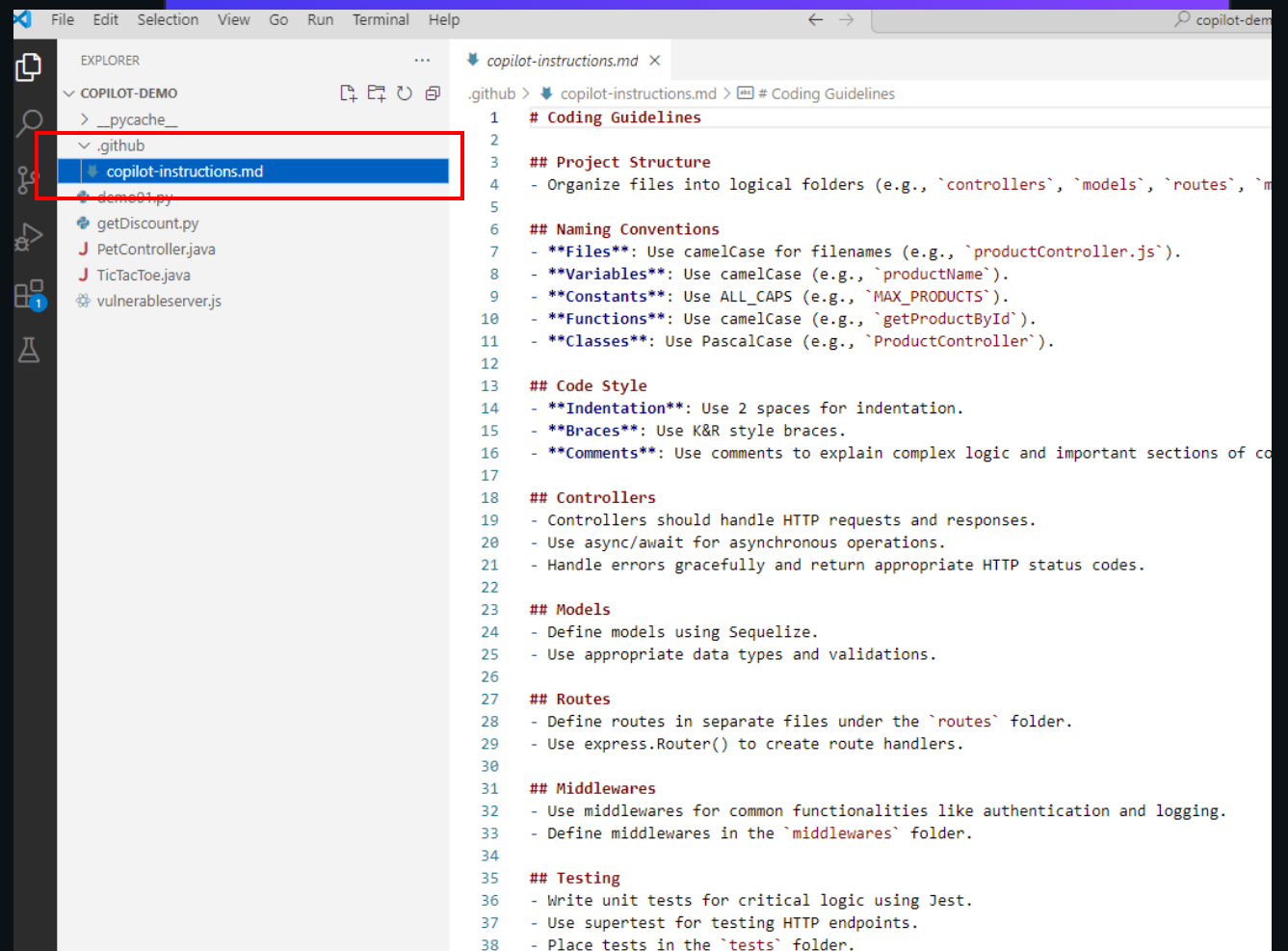


VS Code에서 copilot-Instructions.md

Copilot에게 코드 제안에 대한
규칙을 설정

.github/copilot-instructions.md

Copilot이 코드 제안시에 이 규칙을
따라 코드 제안



```
File Edit Selection View Go Run Terminal Help
EXPLORER
  COPILOT-DEMO
    > __pycache__
    > .github
      copilot-instructions.md
      demo01.py
    getDiscount.py
    PetController.java
    TicTacToe.java
    vulnerableserver.js
  copilot-instructions.md x
.github > copilot-instructions.md > # Coding Guidelines
1  # Coding Guidelines
2
3  ## Project Structure
4  - Organize files into logical folders (e.g., `controllers`, `models`, `routes`, `m
5
6  ## Naming Conventions
7  - **Files**: Use camelCase for filenames (e.g., `productController.js`).
8  - **Variables**: Use camelCase (e.g., `productName`).
9  - **Constants**: Use ALL_CAPS (e.g., `MAX_PRODUCTS`).
10 - **Functions**: Use camelCase (e.g., `getProductById`).
11 - **Classes**: Use PascalCase (e.g., `ProductController`).
12
13 ## Code Style
14 - **Indentation**: Use 2 spaces for indentation.
15 - **Braces**: Use K&R style braces.
16 - **Comments**: Use comments to explain complex logic and important sections of co
17
18 ## Controllers
19 - Controllers should handle HTTP requests and responses.
20 - Use async/await for asynchronous operations.
21 - Handle errors gracefully and return appropriate HTTP status codes.
22
23 ## Models
24 - Define models using Sequelize.
25 - Use appropriate data types and validations.
26
27 ## Routes
28 - Define routes in separate files under the `routes` folder.
29 - Use express.Router() to create route handlers.
30
31 ## Middlewares
32 - Use middlewares for common functionalities like authentication and logging.
33 - Define middlewares in the `middlewares` folder.
34
35 ## Testing
36 - Write unit tests for critical logic using Jest.
37 - Use supertest for testing HTTP endpoints.
38 - Place tests in the `tests` folder.
```

VS Code에서 Prompt instructions 설정

Copilot에게 재사용 가능한 프롬프트 instruction 파일(.md)을 제공

- 코드 생성: 컴포넌트, 테스트, migration등에 대한 재사용 프롬프트 제공(예: React forms, API mocks)
- 도메인 전문지식 : 특정 도메인에 대한 전문적인 정보 제공 (예: 보안 프랙티스, 규정 준수확인 등)
- 팀 협업: 문서 패턴과 가이드라인 제공 (참조문서와 스펙제공)
- 온보딩: 복잡한 절차, 혹은 프로젝트의 특정한 패턴들에 대한 step-by-step 가이드

react-form.prompt.md

Your goal is to generate a new React form component. Copy

Ask for the form name and fields if not provided.

Requirements for the form:

- * Use form design system components: [design-system/Form.md](../docs/design-system/Form.md)
- * Use `react-hook-form` for form state management:
- * Always define TypeScript types for your form data
- * Prefer *uncontrolled* components using register
- * Use `defaultValues` to prevent unnecessary rerenders
- * Use `yup` for validation:
- * Create reusable validation schemas in separate files
- * Use TypeScript types to ensure type safety
- * Customize UX-friendly validation rules

security-api.prompt.md

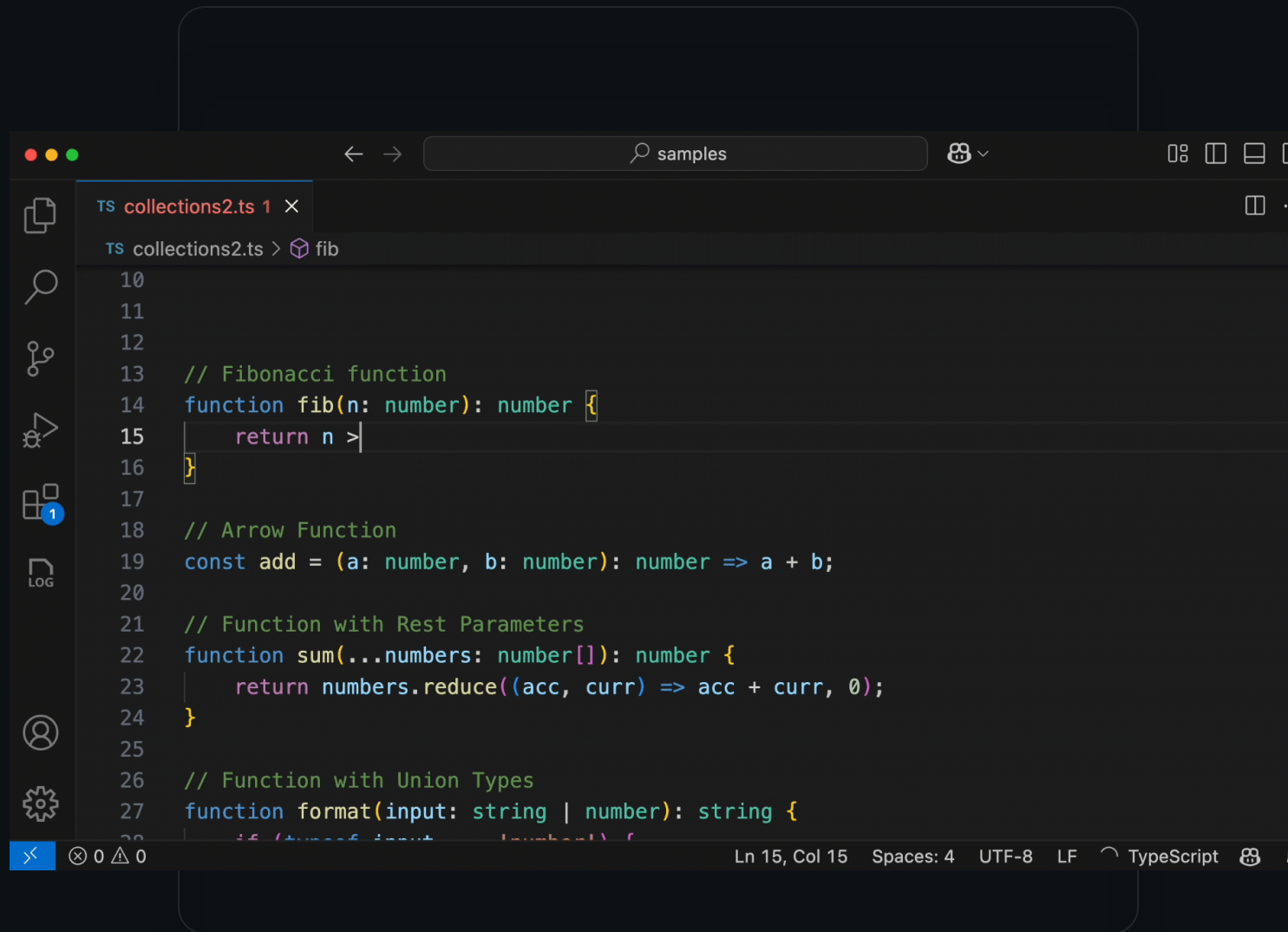
Secure REST API review: Copy

- * Ensure all endpoints are protected by authentication and authorization
- * Validate all user inputs and sanitize data
- * Implement rate limiting and throttling
- * Implement logging and monitoring for security events

Next Edit suggestions

변경되는 내용을 기반으로
다음에 이어질 수정 사항을
자동으로 제안.

코드, 주석, 테스트 등 다양한
내용을 제안



 코드

 GitHub Copilot

코딩 에이전트

Assignees



Assign up to 10 people



 Type or choose a user

✓  **Copilot** Your AI pair programmer

 **rodbotas6** Rodrigo Botas

 **chandiverse** Chandr

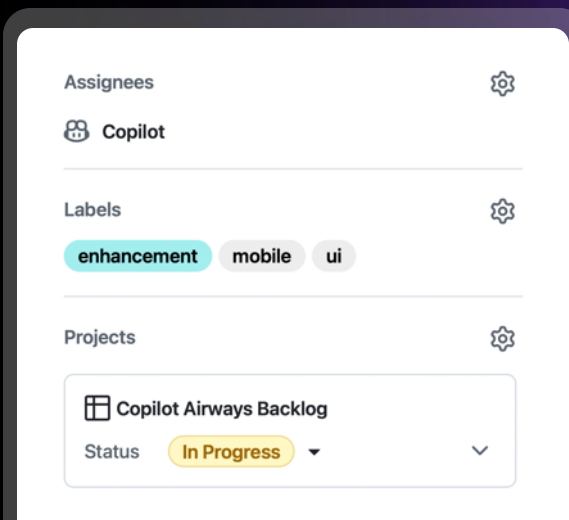
 **semyonrush** Semyo

 **magnusflare** Ólaf

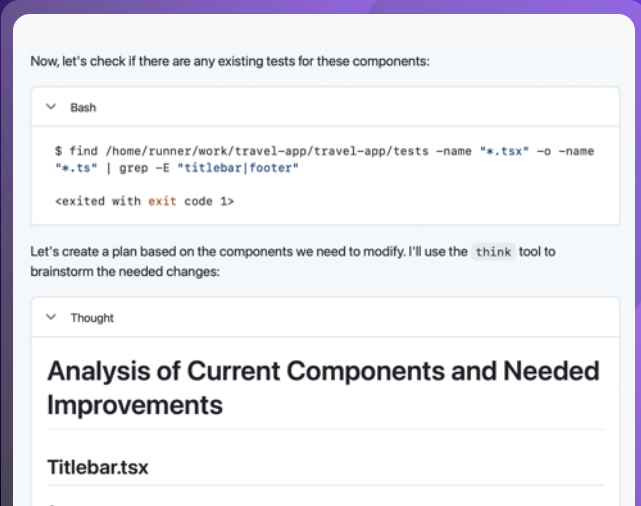


소개/

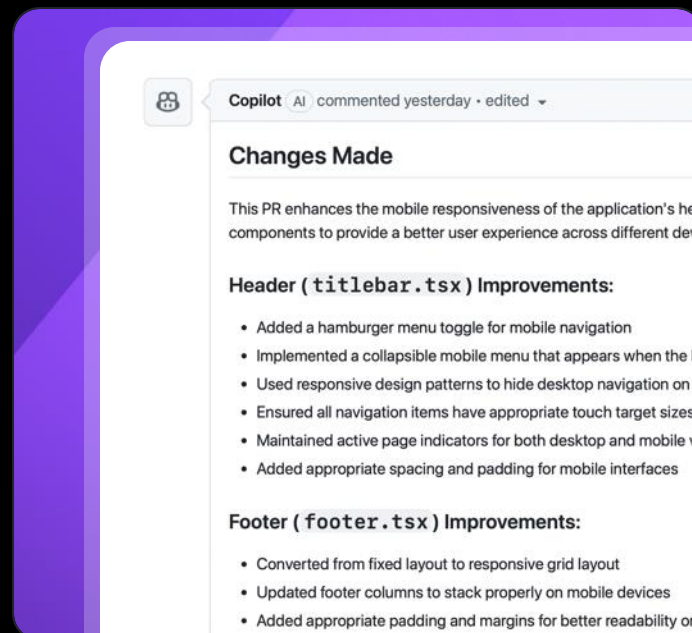
Copilot 코딩 에이전트



Copilot에 작업 제공



에이전트의 작업 상황 추적



에이전트의 작업 검토



에이전트에는 보호 장치가 있습니다.



- ① 작업을 실행하기 전에 에이전트가 작성한 코드 검토
- ② 담당자가 Copilot에서 만든 PR을 승인할 수 없음
- ③ 단일 브랜치로 제한된 에이전트 푸시 액세스(copilot/ 브랜치)

✓ 검증

MCP 서버는 AI가
생성한 비밀을 자
동으로 차단합니
다.

Create an issue with info from data.txt



GitHub Copilot

> Ran Create issue ❌

Tool call blocked. Secret detected in data.txt

Bypass to proceed: [github.com/unblock-secret/...](https://github.com/unblock-secret/)

GitHub Advanced Security

Publicly leaked active secret

d3efc47d778cd3eb006e93731cee3769-1f1bd6a9-9ff949fa

Public leaks

- 📄 path/to/the/specific/file/on/repository-0.ext c0af16f
- 📄 another/path/to/repository-0.ext c0af16f

시크릿 검사(API Key 등)

```
→ ~/my_project git:(branch_name) git push
```

```
remote: error GH009: Secrets detected! This push failed.
```

```
remote:
```

```
remote:          GITHUB PUSH PROTECTION
```

```
remote:
```

```
remote: Resolve the following secrets before pushing  
again.
```

Push 보호

Copilot PR Summary

Pull Request에 대한
summary를 Copilot에게 요청

전체 Summary 혹은 아웃라인
요청 가능

The screenshot displays a GitHub Pull Request titled "Update codeql-analysis.yml #28". The interface includes a header with the title and a green "Open" button. Below the header, there are statistics: "Conversation 0", "Commits 1", "Checks 1", and "Files changed 1". A comment by "son7211" dated "Oct 23, 2024" is visible. The comment area has a "Write" tab and a "Preview" tab. A "Text Completion v1.0.0" button is present. On the right side, a "Generate" panel is open, showing options for "Summary" (Generate a summary of the changes in this pull request.) and "Outline" (Generate a list of the most important changed files in this pull request.). Below the "Generate" panel, there is a "Settings" section with a "Text complete" option, which is currently set to "Preview". At the bottom of the interface, a green checkmark icon and the text "All checks have passed" are visible.

Update codeql-analysis.yml #28

Open son7211 wants to merge 1 commit into master from son7211-patch-2

Conversation 0 Commits 1 Checks 1 Files changed 1

son7211 commented on Oct 23, 2024

Write Preview

Leave a comment

Text Completion v1.0.0

Update codeql-analysis.yml

Generate

Summary

Generate a summary of the changes in this pull request.

Outline

Generate a list of the most important changed files in this pull request.

Settings

Text complete Preview

Text completions while typing

All checks have passed

Copilot summaries for discussions & issues

AI가 제공해 주는
요약을 통해 편리하고
신속하게 긴 문맥을
이해

주요 포인트와 문맥을
편리하게 이해하도록
하여, 개발팀간 협업을
더욱 원활하게 하며
생산성을 높여 줌

The screenshot shows a GitHub issue page for "Implement different sorting logic for add context quick pick #246541". The issue is labeled "Feature" and "Open". The author is hawkticehurst, who opened it last week. The issue text describes a problem with alphabetical sorting in the add context quick pick and proposes an alternative "pinned list" of options followed by a "most recently used" sort. A list of options is provided: "Codebase", "Fetch Web Page", "Find Test Files", "Folder", "Git Changes", "Problem", "Terminal Last Command", "Terminal Selection", "Test Failure", and "... Insert other tools/resources sorted by most recently used ...". The issue also mentions a "Stretch goal" of implementing an AI-powered recommendation based on open files and context.

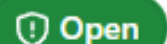
On the right side of the issue page, there is a sidebar with "Assignees" (jrieken, roblourens), "Labels" (chat, chat-input), "Type" (Feature), "Projects" (VS Code Design), "Milestone" (April 2025), "Relationships" (None yet), and "Development" (Code with Copilot).

A red box highlights a Copilot summary overlay on the right side of the page. The summary is titled "이슈 요약" (Issue Summary) and includes the following information:

- 제목: Implement different sorting logic for add context quick pick
- 작성자: hawkticehurst
- 상태: Open
- 생성일: 7 days ago
- 업데이트: 4 days ago
- 라벨: ux, chat, chat-input
- 마일스톤: April 2025
- 담당자: roblourens, jrieken

At the bottom of the summary, there is a "Ask Copilot" button.

Server-side request forgery

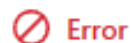
 Open in master on Oct 23, 2024

 Copilot Autofix  AI generated a fix 5 months ago

To fix the SSRF vulnerability, we need to validate the `host` header before using it to construct the request. We need to maintain a list of authorized hosts and ensure that the `host` header matches one of these authorized hosts. This approach will prevent attackers from injecting arbitrary hosts into the request.

1. Create a list of authorized hosts.
2. Check if the `host` header is in the list of authorized hosts.
3. If the `host` is not authorized, handle the error appropriately (e.g., log the incident and return an error).

```
...set/src/main/java/org/owasp/webgoat/password_reset/ResetLinkAssignmentForgotPassword.java
```



Error

Server-side request forgery

Potential server-side request forgery due to a `user-provided value`.

Directly incorporating user input into an HTTP request without validating the input can facilitate server-side request forgery. If an attacker can control these attacks, the server may be tricked into making a request and interacting with an attacker-controlled server.

Show paths

... @@ -35,2 +35,3 @@



 johnjohncom/testvulner

이 보안 문제를 수정하려면 어떻게 하지?



보안 문제: Server-Side Request Forgery (SSRF)

문제 설명:

사용자 입력을 검증 없이 HTTP 요청에 직접 포함시키는 경우, 서버가 공격자가 제어하는 서버와 통신하도록 조작될 수 있습니다. 이는 Server-Side Request Forgery(SSRF) 공격을 초래할 수 있습니다.

취약 코드 위치:

파일:

```
webgoat-lessons/password-reset/src/main/java/org/owasp/webgoat/password_reset/ResetLinkAssignmentForgotPassword.java
```

라인:

Ask Copilot



Add /user endpoint #11

Open



son7211 opened on Aug 30, 2024

- add /user endpoint for user to show each user's profile
- /user/order endpoint for user's order information (product and the number of product

Create sub-issue



son7211 self-assigned this on Aug 30, 2024



son7211 added a commit that references this issue

Add /user endpoint ...



son7211 mentioned this on Sep 12, 2024

Add /user endpoint #11



johnjohncom/demo-onlinestore-s...

이 기능을 추가하려면 어떻게 해야 할지 제안해줘



해당 URL에서 제공된 이슈

(<https://github.com/johnjohncom/demo-onlinestore-server/issues/11>)는 특정 이슈에 대한 내용으로 보입니다. 이를 기반으로 추가 개발 또는 기능 구현 방법을 제안하려면, 먼저 이슈의 세부 내용을 확인해야 합니다. 지금 바로 이슈의 내용을 확인하겠습니다. Issue #11 in this repository suggests adding the following endpoints:

1. **/user** endpoint: To show each user's profile.
2. **/user/order** endpoint: To display a user's order information, including the product and the number of products.

Steps to Implement:

1. Define the Routes:

Copilot is responding...

Steps to Implement:

1. Define the Routes:

- Use your server framework (e.g., Express.js for Node.js) to define new routes for `/user` and `/user/order`.

2. Database Integration:

- Ensure your database has the necessary tables and fields for user profiles and order information.
- For `/user`, retrieve user profile data such as name, email, etc.
- For `/user/order`, fetch the order details linked to the user, including product names and quantities.

3. Controller Logic:

- Create functions in your controller to handle the logic for fetching and formatting data for the endpoints.

Copilot (AI) reviewed just now

common/models/application.js

```
24 + }  
25 +  
26 +   useEffect(() => {  
27 +     const response = await fetch(basePageDataUrl)
```

Copilot (AI)

The fetch call is not awaited. This can lead to unexpected behavior.
Use `await fetch(basePageDataUrl)` instead.

Suggested change

```
27 -   const response = fetch  
27 +   const response = await
```



Request up to 100 reviewers

Q Type or choose a user

✓ Copilot

fernflare Fernanda Sarmento

rodbotas6 Rodrigo Botas

Copilot 코드 검토

기다릴 필요 없는 코드 리뷰 🔥

Copilot으로 버그를 발견하고, 오타를
수정하고, 오류 처리를 개선하는 등 더
빠른 머지로의 여정을 시작하세요!

Code review guideline 설정

The screenshot shows the GitHub interface for configuring code review guidelines. The top navigation bar includes links for Code, Issues, Pull requests (1), Actions, Projects, Wiki, Security, Insights, and Settings. The main heading is "Edit coding guideline" with a "Preview" button. Below this, the "Definition" section contains instructions to be clear and specific about what Copilot should look for, with a link to "Learn more in the docs." The "Name" field is set to "kotlin". The "Description" field contains two bullet points: "- 모든 함수이름은 'fun_' 으로 시작해야 한다" and "- 모든 함수는 doc 설명을 가져야 한다." The "Sample" section shows a Kotlin code snippet for a calculator. The "Generate from description" button is visible. On the right, a "Run" button and a "Save guideline" button are present. A red box highlights the "Edit coding guideline" header and the "Description" field. Another red box highlights the "Sample" code and the "Generate from description" button. A third red box highlights the "Line 1" and "Line 9" suggestions in the right sidebar.

johnjohncom / kotlin-calculator

Code Issues Pull requests 1 Actions Projects Wiki Security Insights Settings

← Edit coding guideline Preview Run Save guideline

Definition

Be clear and specific about what Copilot should look for.
[Learn more in the docs.](#)

Name *

kotlin

Description *

- 모든 함수이름은 'fun_' 으로 시작해야 한다
- 모든 함수는 doc 설명을 가져야 한다.

Sample Generate from description

```
1 private fun calculate(request: ArithmeticRequest): Double = when
  (request.operation) {
2     "add" -> request.operand1 + request.operand2
3     "subtract" -> request.operand1 - request.operand2
4     "multiply" -> request.operand1 * request.operand2
5     "divide" -> request.operand1 / request.operand2
6     "power" -> Math.pow(request.operand1, request.operand2)
7     else -> throw IllegalArgumentException("Invalid operation:
  ${request.operation}")
8 }
9
10 fun handle(req: Request): Response {
11     val operation = operationLens(req)
12     val operand1 = operand1Lens(req).toDouble()
13     val operand2 = operand2Lens(req).toDouble()
14     val arithmeticRequest = ArithmeticRequest(operation, operand1,
  operand2)
15     println("arithmeticRequest: $arithmeticRequest")
16     return try {
17         val result = calculate(arithmeticRequest)
```

Line 1
Function name must start with 'fun_'. Rename to match the required prefix. ```suggestion private fun fun_calculate(request: ArithmeticRequest): Double = when (request.operation) { ```

Line 1
Function must include a doc description per coding guidelines.

Line 10
Function name must start with 'fun_'. Rename to match the required prefix.

Line 9
Function must include a doc description per coding guidelines. ```suggestion /** * Handles the incoming request and performs the specified arithmetic operation. * * @param req The incoming request containing the operation and operands. * @return A Response object containing the result of the arithmetic operation or an error message. */ ```

🛡️ Copilot 자동 수정

몇 초 만에 수정 사항 적용

취약점을 수정하는 데 소요되는 시간을
줄이고 기능 구축에 더 많은 시간을 할애
합니다.

github-advanced-security bot

The vulnerability in the code is due to the fact that user-provided input is directly used in HTTP response without any sanitization. This can lead to a cross-site scripting (XSS) attack if the user input contains malicious scripts.

To fix this, we need to sanitize the input before using it in the HTTP response. One way to do this is to use the `escape-html` library, which can escape any special characters.

Copilot Autofix AI

```
14     app.get('/', (req, res) => {  
15         -   res.send('Hello ${req.query.name}!');  
15         +   res.send('Hello ${escape(req.query.name)}!');  
16     })
```

Copilot 자동 수정

github-advanced-security bot

The vulnerability in the code is due to the fact that user-provided input is directly used in HTTP response without any sanitization. This can lead to a cross-site scripting (XSS) attack if the user input contains malicious scripts.

To fix this, we need to sanitize the input before using it in the HTTP response. One way to do this is to use the `escape-html` library, which can escape any special characters.

Copilot Autofix AI

```
14 app.get( '/', (req, res) => {  
15   -   res.send('Hello ${req.query.name}!');  
15   +   res.send('Hello ${escape(req.query.name)}!');  
16 });
```

실패 시 업데이트



Demo

시나리오 논의

1. RAG 구성

- .md에 룰을 넣어서 결과를 보는 방법?
- 더 정확한 context 정보 사용을 위해서는 > Github Space > instructions.md가 더 많은 context 입력이 가능하지만, Space에 제한된 정보를 입력하여 정확도를 높일 수 있음
- Github space를 사용하더라도, 너무 많은 양의 context를 넣으면 실제 코드의 구조를 제대로 담지 못하기 때문에, 최대한 간결하게 작성해야 성능 향상의 효과를 볼 수 있음
- VS Code에서 바로 Github space를 MCP 서버로 구성할 수도 있음

2. 코드 refactoring ([참고 문서](#))

- 가독성 : 자동 주석, 변수 네이밍 표준화 등
 - “조건을 switch로 다시 작성하고 null 케이스를 포함한 Java 21 문법을 사용하며 문서화도 추가하고 더 나은 함수 이름을 제공하세요.”
- 재사용성 : 공통 로직 추출 (이후 instructions.md로 반영)
 - “반복되는 계산을 함수로 만들어줘”
- 기존 혹은 잠재적 문제점 개선 : 정적 경고 분석으로 수정 코드 제안, 단위 테스트 코드 작성 등



감사합니다